Last Set of Exercises – Not to be submitted! - SOLUTION

---

1. Locality and caching

Consider a computer with the following memory model:

- Size of memory: M = 2^m x n bits
- Byte-addressable memory
- Memory hierarchy has three layers: CPU register – L1 cache – main memory
- L1 cache has a size of 1024 bytes and is defined by the tuple (4, 2, B, 48)
- Initially, before a program starts executing, L1 cache is empty.

Now, imagine we execute a program that consists of the function `printA(…)`, which prints the content of array `A` defined as follows:

```
#define N 2
#define M 3

typedef struct {
    int labs[M];
    int assns[M];
} activity;

activity A[N];
```

This function is called from the `main(…)` function as follows: `printA(A, N, M)`.

Here is (most of) the body of `printA(activity *A, int n, int m)`:

```
for (i = 0; i < n; i++)
  for (j = 0; j < m; j++) {
     printf("A[%d].labs[%d] = %d\n", i, j, A[i].labs[j]);
     printf("A[%d].assns[%d] = %d\n", i, j, A[i].assns[j]);
  }
```

When the first iteration of `printA`'s inner loop executes, two memory accesses take place: the microprocessor sends a `load` request to the first layer of the memory hierarchy requesting the content of `A[0].labs[0]`, then it sends a second `load` request for the content of `A[0].assns[0]` in order to print them. The first memory access is done at memory address `0x6FCD00`.

As you answer the following question, always show your work!

a. What is the size of array `A` in bytes?

As I answer this question, I shall make the following assumption: the question is asking for the size of array `A` in bytes.

Solution:

```
typedef struct {
    int labs[3];
    int assns[3];
} activity;

activity A[2];

array A has 2 x activity = 2 x (3 x labs + 3 x assns)
                         = 2 x (3 x 4 bytes + 3 x 4 bytes)
                         = 2 x (24 bytes) = 48 bytes
```

b. Draw the array `A` has it is laid out in memory along with the memory addresses of each of its elements.

Solution:

```
0x6FCD00 -> A[0].labs[0] -> 011011111100110 10 0000000
0x6FCD04 -> A[0].labs[1] -> 011011111100110 10 0000100
0x6FCD08 -> A[0].labs[2] -> 011011111100110 10 0001000
0x6FCD0C -> A[0].assns[0]
0x6FCD10 -> A[0].assns[1]
0x6FCD14 -> A[0].assns[2] -> 011011111100110 10 0010100
0x6FCD18 -> A[1].labs[0]
0x6FCD1C -> A[1].labs[1]
0x6FCD20 -> A[1].labs[2]
0x6FCD24 -> A[1].assns[0]
0x6FCD28 -> A[1].assns[1] -> 011011111100110 10 0101000
0x6FCD2C -> A[1].assns[2]
```

Array in memory (with memory addresses):

Element # (each element is an **int**):

1	2	3	4	5	6	7	8	9	10	11	12
A[0].labs[0]	A[0].labs[1]	A[0].labs[2]	A[0].assns[0]	A[0].assns[1]	A[0].assns[2]	A[1].labs[0]	A[1].labs[1]	A[1].labs[2]	A[1].assns[0]	A[1].assns[1]	A[1].assns[2]
0x6FCD00	D04	D08	D0C	D10	D14	D18	D1C	D20	D24	D28	D2C

c. What is the value of C (in bytes), B (***block*** size in bytes), b (***block offset*** in bits), s (***set index*** in bits), t (***cache tag*** in bits)?

Solution:

- C (in bytes) = S x E x B = 1024 bytes

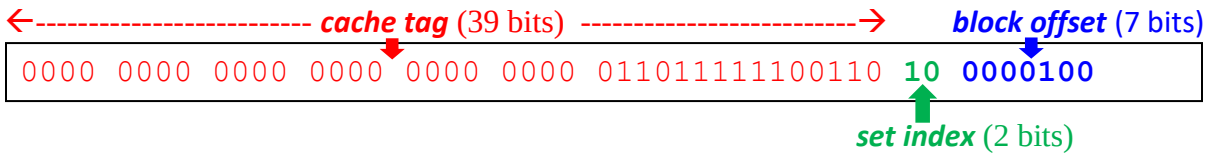
From: L1 cache has a size of 1024 bytes and is defined by the tuple (4, 2, B, 48)

- Since S = 4 (2^2) sets, then s (***set index*** in bits) = 2 bits
- E = 2 lines per set
- B (***block*** size in bytes) = 1024 bytes/(4 sets x 2 lines per set)
  = 128 bytes per line (or block in a line)
- Since B = 128 (2^7) bytes/line, then b (***block offset*** in bits) = 7 bits
- t (***cache tag*** in bits) = 48 bites (m) – 7 bits (b) – 2 bits (s) = 39 bits

d. Expand the memory address of the second element of `A` (i.e., `A[0].labs[1]`) as a bit pattern in the following template (we have already started doing so) and clearly label these three sections: the ***block offset***, the ***set index*** and the ***cache tag***, as well as the size (in bits) of each of these sections:

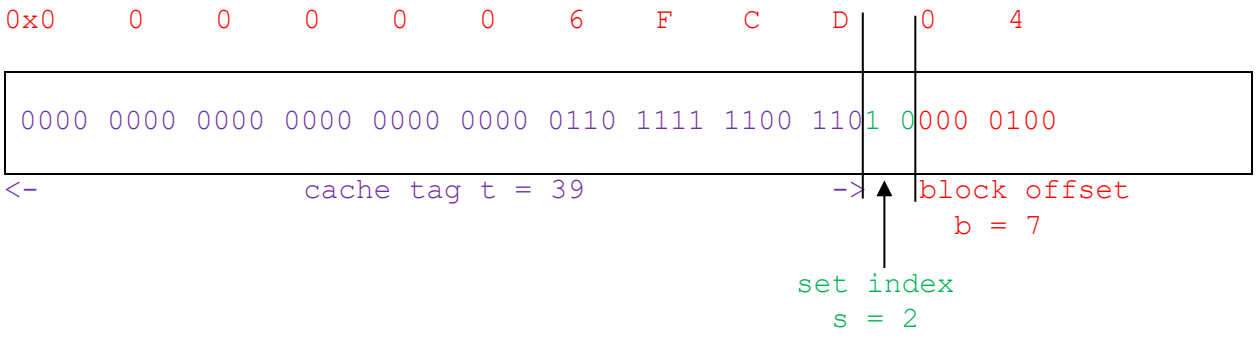
Solution:

The second element of `A` is `A[0].labs[1]` at memeory address `0x6FCD04`
`-> 0110 1111 1100 1101 0000 0100`



OR

Second element of **`A`**: **`A[0].labs[1]`**



e. Considering the above description of the execution of the function `printA(…)`, complete the following table for the given specific elements of array `A`:

Element of array A	**Memory address of this element**	**When loaded into cache, in which set of L1 cache would this element be copied?**	**When `printA(…)` executes and attempts to read this element from L1 cache, does it produce a *cache hit* or a *cache miss*?**	**If reading this element produces a *cache hit*, at which *block offset* in L1 cache block would we find this element? If reading this element produces a *cache miss*, leave the space blank.**
`A[0].labs[0]`	`0x6FCD00`	Since set index are read as unsigned binary numbers, the sets are numbered: Set 0, Set 1, Set 2, Set 3 Set 2 (10_2)	"Initially, before a program starts executing, L1 cache is empty": so cache is cold => cache miss	
`A[0].labs[1]`	`0x6FCD04`	Set 2 (10_2)	Since A fits in a block: Cache hit	Block offset 4 (0000100_2)
`A[0].assns[2]`	`0x6FCD14`	Set 2 (10_2)	Cache hit	Block offset 20 (0010100_2)

`A[1].assns[1]`	`0x6FCD28`	Set 2 (10_2)	Cache hit	Block offset 40 (0101000_2)

f. How many times does `printA(…)` access memory, i.e., how many times does the microprocessor send a `load` request to the first layer of the memory hierarchy requesting the content of an element of `A`?

Answer: 12 times since there are 12 elements in array `A`

g. What is the miss rate of `printA(…)`?

Answer:

Miss rate => number of misses/# of memory accesses

=> 1 / 12 (1 miss/ 12 memory accesses)

=> the cache miss occurred during the first memory access since initially, when the program starts executing, L1 cache is empty.

---

2. Locality and caching

Consider a computer with the following memory model:

- Size of memory: M = 2^m x n bits
- Byte-addressable memory
- Memory hierarchy has three layers: CPU register – L1 cache – main memory
- L1 cache has a size of C bytes and is defined by the tuple (4, 2, 128, 48)
- Initially, before a program starts executing, L1 cache is empty.

Now, imagine we execute a program that consists of the function `printA(…)`, which prints the content of array `A` defined as follows:

```
#define N 8
#define M 4

typedef struct {
    int labs[M];
    int assns[M];
} activity;

activity A[N];
```

This function is called from the `main(…)` function as follows: `printA(A, N, M)`.

Here is (most of) the body of `printA(activity *A, int n, int m)`:

```
for (i = 0; i < n; i++)
  for (j = 0; j < m; j++) {
     printf("A[%d].labs[%d] = %d\n", i, j, A[i].labs[j]);
     printf("A[%d].assns[%d] = %d\n", i, j, A[i].assns[j]);
  }
```

When the first iteration of `printA`'s inner loop executes, two memory accesses take place: the microprocessor sends a `load` request to the first layer of the memory hierarchy requesting the content of `A[0].labs[0]`, then it sends a second `load` request for the content of `A[0].assns[0]` in order to print them. The first memory access is done at memory address `0x6FCD00`.

As you answer the following question, always show your work!

a. What is the size of array `A` in bytes?

As I answer this question, I shall make the following assumption: the question is asking for the size of array `A` in bytes.

Solution:

```
typedef struct {
    int labs[4];
    int assns[4];
} activity;

activity A[8];

array A has 8 x activity = 8 x (4 x labs + 4 x assns)
                         = 8 x (4 x 4 bytes + 4 x 4 bytes)
                         = 8 x (32 bytes) = 256 bytes
```

b. Draw the array `A` as it is laid out in memory along with the memory addresses of each of its elements.

Solution:

```
0x6FCD00 -> A[0].labs[0] -> 011011111100 110 10 0000000
0x6FCD04 -> A[0].labs[1]
0x6FCD08 -> A[0].labs[2]
0x6FCD0C -> A[0].labs[3] -> 011011111100 110 10 0001100
0x6FCD10 -> A[0].assns[0]
0x6FCD14 -> A[0].assns[1]
0x6FCD18 -> A[0].assns[2]
0x6FCD1C -> A[0].assns[3]
```

```
0x6FCD20 -> A[1].labs[0]
0x6FCD24 -> A[1].labs[1]
0x6FCD28 -> A[1].labs[2]
0x6FCD2C -> A[1].labs[3]
0x6FCD30 -> A[1].assns[0]
0x6FCD34 -> A[1].assns[1]
0x6FCD38 -> A[1].assns[2] -> 011011111100 110 10 0111000
0x6FCD3C -> A[1].assns[3]
0x6FCD40 -> A[2].labs[0] -> 011011111100 110 10 1000000
0x6FCD44 -> A[2].labs[1]
0x6FCD48 -> A[2].labs[2]
0x6FCD4C -> A[2].labs[3]
0x6FCD50 -> A[2].assns[0]
0x6FCD54 -> A[2].assns[1]
0x6FCD58 -> A[2].assns[2]
0x6FCD5C -> A[2].assns[3]
0x6FCD60 -> A[3].labs[0]
0x6FCD64 -> A[3].labs[1]
0x6FCD68 -> A[3].labs[2]
0x6FCD6C -> A[3].labs[3]
0x6FCD70 -> A[3].assns[0]
0x6FCD74 -> A[3].assns[1]
0x6FCD78 -> A[3].assns[2] -> 011011111100 110 10 1111000
0x6FCD7C -> A[3].assns[3]
0x6FCD80 -> A[4].labs[0] -> 011011111100 110 11 0000000
0x6FCD84 -> A[4].labs[1]
0x6FCD88 -> A[4].labs[2]
0x6FCD8C -> A[4].labs[3]
0x6FCD90 -> A[4].assns[0] -> 011011111100 110 11 0010000
0x6FCD94 -> A[4].assns[1]
0x6FCD98 -> A[4].assns[2]
0x6FCD9C -> A[4].assns[3]
0x6FCDA0 -> A[5].labs[0]
0x6FCDA4 -> A[5].labs[1] -> 011011111100 110 11 0100100
0x6FCDA8 -> A[5].labs[2]
0x6FCDAC -> A[5].labs[3]
0x6FCDB0 -> A[5].assns[0]
0x6FCDB4 -> A[5].assns[1]
0x6FCDB8 -> A[5].assns[2]
0x6FCDBC -> A[5].assns[3]
0x6FCDC0 -> A[6].labs[0]
0x6FCDC4 -> A[6].labs[1]
0x6FCDC8 -> A[6].labs[2] -> 011011111100 110 11 1001000
0x6FCDCC -> A[6].labs[3]
0x6FCDD0 -> A[6].assns[0]
0x6FCDD4 -> A[6].assns[1]
0x6FCDD8 -> A[6].assns[2]
0x6FCDDC -> A[6].assns[3]
```

```
0x6FCDE0 -> A[7].labs[0]
0x6FCDE4 -> A[7].labs[1]
0x6FCDE8 -> A[7].labs[2]
0x6FCDEC -> A[7].labs[3]
0x6FCDF0 -> A[7].assns[0]
0x6FCDF4 -> A[7].assns[1]
0x6FCDF8 -> A[7].assns[2]
0x6FCDFC -> A[7].assns[3] -> 011011111100 110 11 1111100
```

c. What is the value of C (in bytes), B (***block*** size in bytes), b (***block offset*** in bits), s (***set index*** in bits), t (***cache tag*** in bits)?

Solution:

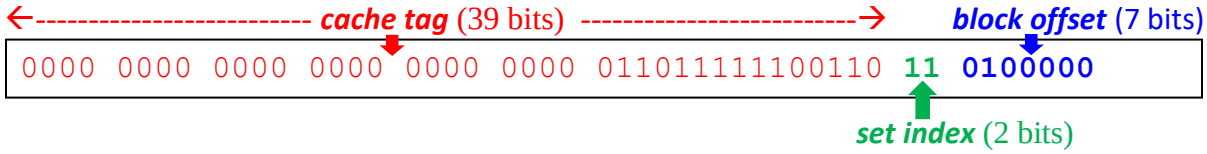
Since: L1 cache has a size of C bytes and is defined by the tuple (4, 2, 128, 48)

- B (***block*** size in bytes) = 128 (2^7) bytes per line
- b (***block offset*** in bits) = 7 bits
- S = 4 (2^2) sets
- E = 2 lines per set
- s (***set index*** in bits) = 2 bits
- t (***cache tag*** in bits) = 48 bites (m) – 7 bits (b) – 2 bits (s) = 39 bits
- C (in bytes) = S x E x B = 4 sets x 2 lines per set x 128 bytes per line = 1024 bytes

d. Expand the memory address of the element of `A` called `A[5].labs[0]` as a bit pattern in the following template (we have already started doing so) and clearly label these three sections: the ***block offset***, the ***set index*** and the ***cache tag***, as well as the size (in bits) of each of these sections:

Solution:

`A[5].labs[0]` -> `0x6FCDA0`



e. Considering the above description of the execution of the function `printA(…)`, complete the following table for the given specific elements of array `A`:

Element of array A	Memory address of this element	When loaded into cache, in which set of L1 cache would this element be copied?	When `printA(…)` executes and attempts to read this element	If reading this element produces a *cache hit*, at which *block offset* in L1

			from L1 cache, does it produces a *cache hit* or a *cache miss*?	**cache block would we find this element? If reading this element produces a *cache miss*, leave the space blank.**
`A[0].labs[0]`	`0x6FCD00`	Since set index are read as unsigned binary numbers, the sets are numbered: Set 0, Set 1, Set 2, Set 3 Set 2 (10_2)	"Initially, before a program starts executing, L1 cache is empty": so cache is cold => cache miss	
`A[0].labs[3]`	`0x6FCD0C`	Set 2 (10_2)	Cache hit	Block offset 12 (0001100_2)
`A[1].assns[2]`	`0x6FCD38`	Set 2 (10_2)	Cache hit	Block offset 56 (0111000_2)
`A[2].labs[0]`	`0x6FCD40`	Set 2 (10_2)	Cache hit	Block offset 64 (1000000_2)
`A[3].assns[2]`	`0x6FCD78`	Set 2 (10_2)	Cache hit	Block offset 120 (1111000_2)
`A[4].labs[0]`	`0x6FCD80`	Set 3 (11_2)	Cache miss Only half of the array fits	

			in cache L1 at a time.	
`A[4].assns[0]`	`0x6FCD90`	Set 3 (11_2)	Cache hit	Block offset 16 (0010000_2)
`A[5].labs[1]`	`0x6FCDA4`	Set 3 (11_2)	Cache hit	Block offset 36 (0100100_2)
`A[6].labs[2]`	`0x6FCDC8`	Set 3 (11_2)	Cache hit	Block offset 72 (1001000_2)
`A[7].assns[3]`	`0x6FCDFC`	Set 3 (11_2)	Cache hit	Block offset 124 (1111100)

f. How many times does `printA(…)` access memory, i.e., how many times does the microprocessor send a `load` request to the first layer of the memory hierarchy requesting the content of an element of `A`?

Solution: 64 times, i.e., the number of elements in `A`

g. What is the miss rate of `printA(…)`?

Solution: 2 elements/64 = 1/32

Since only half of array `A` fits in a block of L1 cache, two `load` requests will miss when reading all elements of array `A`.